

TEXT EDITOR UNDO/REDO SYSTEM

เอกสารประกอบการนำเสนอ - กลุ่มที่ 3 (Snapshot vs Command Method)

สไลด์ที่ 1: ข้อมูลกลุ่มและหัวข้อโครงงาน

- **หัวข้อ:** การพัฒนาระบบ Undo/Redo ใน Text Editor ด้วยเทคนิค Snapshot และ Command Method
- **กลุ่มที่ 3:** [ใส่ชื่อสมาชิกในกลุ่มทุกคนพร้อมรหัสนักศึกษา]
- **รายวิชา:** การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack (ภาษา Java)

สไลด์ที่ 2: Case Story & Problem Analysis

- **Case Story:** ระบบ Text Editor ที่รองรับ Action 3 รูปแบบ ได้แก่ INSERT, DELETE, REPLACE
- **เงื่อนไขหลัก:** เมื่อมีการใช้งาน Undo แล้วทำการกระทำใหม่ (New Action) ประวัติใน Redo Stack จะต้องถูกล้างทิ้งทั้งหมด
- **การวิเคราะห์ข้อมูล:**
 - **Input:** คำสั่งแก้ไขข้อความ (ประกอบด้วย Action ID, Type, Position, Old Text, New Text)
 - **Output:** สถานะข้อความล่าสุด และการจัดการ Undo/Redo อย่างถูกต้อง
 - **Constraints & Edge Cases:** ข้อความว่าง, ตำแหน่ง Index ไม่ถูกต้อง, การพยายาม Undo ขณะ Stack ว่าง

สไลด์ที่ 3: Algorithm A (Snapshot Method)

- **แนวคิดหลัก:** บันทึกข้อความทั้งฉบับ (Full Text Snapshot) ลงใน Stack ก่อนหรือหลังการแก้ไขทุกครั้ง
- **ข้อดี:** พัฒนาง่าย Logic ไม่ซับซ้อน การ Undo/Redo ทำได้โดยการ Pop ข้อมูลเก่ามาแทนที่ข้อความปัจจุบันทั้งหมด
- **ข้อจำกัด:** ใช้พื้นที่จัดเก็บมหาศาล ($O(n)$ ต่อหนึ่ง Action) ไม่เหมาะกับเอกสารขนาดใหญ่

สไลด์ที่ 4: Algorithm B (Command / Delta Method)

- **แนวคิดหลัก:** เก็บเฉพาะข้อมูลส่วนที่มีการเปลี่ยนแปลง (Delta) และประมวลผลย้อนกลับด้วย Inverse Operation
- **ตัวอย่าง Inverse Operation:**
 - **INSERT:** ลบข้อความที่เพิ่งแทรกออกไป
 - **DELETE:** นำข้อความที่ถูกลบแทรกกลับเข้าไปที่ตำแหน่งเดิม
 - **REPLACE:** นำ Old Text เข้าไปแทนที่ New Text
- **ข้อดี:** ประหยัดหน่วยความจำอย่างมาก
- **ข้อจำกัด:** Logic ในการคำนวณตำแหน่งและข้อความย้อนกลับมีความซับซ้อน

สไลด์ที่ 5: ตัวอย่างการทำงานแบบ Step-by-Step

- **เหตุการณ์จำลอง:** ข้อความเริ่มต้น "Hello" → แทรก "World" ที่ Index 5 → กด Undo
- **การทำงานของ Snapshot:**
UndoStack เก็บ ["Hello", "HelloWorld"] → เมื่อ Undo ถึง "Hello" ออกมาใช้
- **การทำงานของ Command:**
UndoStack เก็บ [INSERT, Pos:5, Text:"World"] → เมื่อ Undo จะเรียกใช้คำสั่งลบความยาว 5 ตัวอักษรที่ Index 5

สไลด์ที่ 6: Correctness Argument (ความถูกต้อง)

- **Precondition:** ข้อความดั้งเดิมและ Stack อยู่ในสภาวะที่ถูกต้องและเชื่อมโยงกัน
- **Invariant:** สำหรับทุกการกระทำ A การประมวลผลย้อนกลับด้วย Inverse Operation จะคืนค่าข้อความกลับสู่สถานะก่อนกระทำ A เสมอ
- **Postcondition:** ระบบสามารถ Undo/Redo กลับไปกลับมาได้ 100% โดยที่ข้อมูลไม่สูญหายและไม่มีข้อผิดพลาดด้าน Index
- **Termination:** อัลกอริทึมทำงานจบในจำนวนรอบที่แน่นอน ($O(1)$ สำหรับการย้อนค่าใน Command)

สไลด์ที่ 7: การวิเคราะห์ Big-O Complexity

Operation / Metric	Snapshot Method	Command (Delta) Method
Undo / Redo Time	$O(n)$ (เมื่อ Copy ข้อความยาว n)	$O(m)$ (ขึ้นกับความยาว Delta m)
Space Complexity (per Action)	$O(n)$	$O(m)$
Total Auxiliary Space (k Actions)	$O(k \times n)$	$O(k \times m)$

สไลด์ที่ 8: Class Diagram & Java Implementation

- **โครงสร้างหลัก (Classes):** Main, TextDocument, ActionCommand, SnapshotManager, CommandManager
- **เทคนิคใน Java:**
 - ใช้ `Deque<T> stack = new ArrayDeque<>()` สำหรับการประมวลผล Stack เพื่อประสิทธิภาพที่สูงที่สุด
 - ใช้ `StringBuilder` ในคลาส `TextDocument` เพื่อลด Overhead ในการเปลี่ยนแปลงสตริง แทนการต่อสตริงด้วย `String` ปกติ

สไลด์ที่ 9: Test Cases & Edge Cases

- **TC01 - TC03:** ทดสอบการทำงานปกติของการ Insert, Delete, Replace ตามลำดับ แล้วกดย้อนกลับ
- **TC04 - TC06:** ทดสอบการทำงานต่อเนื่อง: Undo หลายครั้ง, Redo หลายครั้ง, และ "Undo แล้วพิมพ์ข้อความใหม่" เพื่อตรวจสอบว่า Redo Stack ถูกลบจริงหรือไม่
- **TC07 - TC08 (Edge Cases):** ส่งข้อความว่าง (Empty String), ส่งคำสั่งแก้ไขที่ Index ตีลบ หรือเกินขอบเขตของข้อความ

สไลด์ที่ 10: ผลการทดลองประสิทธิภาพ (Experimental Results)

- **เงื่อนไขการทดสอบ:** ข้อความขนาดความยาว 100 จนถึง 100,000 ตัวอักษร และจำลองการพิมพ์ (Action) 100 ถึง 10,000 ครั้ง
- **Memory Usage:** Command Method ใช้พื้นที่น้อยกว่า Snapshot Method อย่างชัดเจนในระดับ 10x - 100x เมื่อข้อความใหญ่ขึ้น
- **Execution Time:** สอดคล้องกับทฤษฎี Big-O โดยที่ Snapshot Method เริ่มมีอาการหน่วงอย่างมีนัยสำคัญเมื่อทดสอบกับข้อความยาว 100,000 ตัวอักษร

สไลด์ที่ 11: การอภิปรายผล & สรุปเลือกวิธี

ข้อสรุปของกลุ่ม: เราเลือก **Algorithm B (Command Method)** เป็นโซลูชันที่เหมาะสมที่สุดสำหรับการพัฒนาระบบ Text Editor จริง

- แม้ Algorithm A (Snapshot) จะพัฒนาง่ายกว่าและลดข้อผิดพลาดด้าน Logic ได้ดี แต่ **ความคุ้มค่าด้าน Memory** ของ Algorithm B มีความสำคัญมากกว่าในแอปพลิเคชันยุคปัจจุบัน

สไลด์ที่ 12: Q&A (การเตรียมความพร้อม)

- **คำถาม:** เหตุใดจึงเลือกใช้ StringBuilder แทน String?

ตอบ: String ใน Java เป็น Immutable หากแก้ไขบ่อยจะเกิดการสร้าง Object ใหม่มหาศาล การใช้ StringBuilder จึงช่วยลด Overhead ของ $O(n)$ ได้

- **คำถาม:** ทำไมเมื่อมีการพิมพ์ข้อความใหม่ (New Action) ต้องล้างประวัติใน Redo Stack?

ตอบ: เพื่อป้องกันไม่ให้ Timeline ของประวัติการทำงานเกิดการแตกกิ่ง (Branching) ซึ่งจะทำให้ข้อมูลเก่าที่ Redo ขัดแย้งกับข้อความปัจจุบัน